





SVEUČILIŠTE U ZAGREBU
Fakultet elektrotehnike i računarstva

PARALELNO PROGRAMIRANJE

Poglavlje 05 - Postupak oblikovanja paralelnih algoritama



RAČUNALNI I PROGRAMSKI MODEL

- Računalni model: **MIMD, raspodijeljena memorija**
  - brzina komunikacije ne ovisi o topologiji mreže
  - pristup lokalnoj memoriji je brži od pristupa udaljenoj memoriji (*na drugom računalu*) – *bliže stvarnim računalnim sustavima*
- Programski model: **sustav zadataka i komunikacijskih kanala**
  - paralelni program je **skup zadataka** koji se *mogu* izvoditi **istovremeno** (pod uvjetom da imamo dovoljno procesora i da su zadovoljene međuovisnosti)
  - **zadatak = program koji izvodi lokalne operacije, šalje i prima poruke i stvara nove zadatke**
  - slanje poruke je neblokirajuća, a **primanje je blokirajuće**
  - rezultat izvođenja programa ne smije ovisiti o raspodjeli zadataka po procesorima

PRIMJERI PARALELNIH PROGRAMA

- Metoda konačnih razlika

$$X_i^{(k+1)} = \frac{X_{i-1}^{(k)} + 2X_i^{(k)} + X_{i+1}^{(k)}}{4}$$

  - primjena u CFD (computational fluid dynamics)
    - https://www.ansys.com/simulation-topics/what-is-computational-fluid-dynamics
    - https://www.simscale.com/product/cfd/
    - https://geosynthetic.wordpress.com/2015/10/16/numerical-methods-in-geotechnical-engineering-2-finite-difference-method/
    - https://www.youtube.com/shorts/x-4_jtrOepk
- Uparena međudjelovanja
- Pretraživanje stabla
- Trivijalno paralelni problemi

OBLIKOVANJE PARALELNOG ALGORITMA

- Svojstva koja želimo postići:
  - **istodobnost , skalabilnost, lokalnost, modularnost**
- Faze razvoja paralelnog algoritma:
  1. **Podjela** – dekompozicija problema na zadatke
  2. **Komunikacija** – određivanje minimalne potrebne komunikacije među zadacima

  } istodobnost i skalabilnost

  3. **Aglomeracija** – grupiranje zadataka i kom. kanala iz faze 1 ako to dovodi do poboljšanja performansi
  4. **Pridruživanje** – dodijeljivanje zadataka procesorima

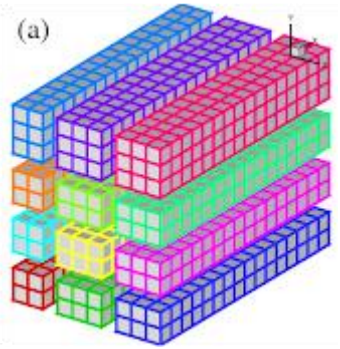
  } lokalnost

  - moguće preklapanje i ponavljanje faza (iterativni razovj)

1. FAZA: PODJELA

- Cilj: postići **sitnozrnatu podjelu bez dupliciranja** podataka ili računanja
  - u praksi nije uvijek moguće, a ponekad ni ne daje najbolja rješenja ➔ rješava se u idućim koracima

- Dva pristupa:



  A. Podjela podataka
    - manje cjeline koje obrađuju različiti zadaci
    - npr. podjela vektora, matrice, tenzora stabla…

  B. Podjela računanja
    - dijeli se izračunavanje, a po potrebi i podaci
    - npr. za neki material se mogu izračunati (1) fiz. opterećenje, (2) toplinska svojstva, (3) vodljivost…

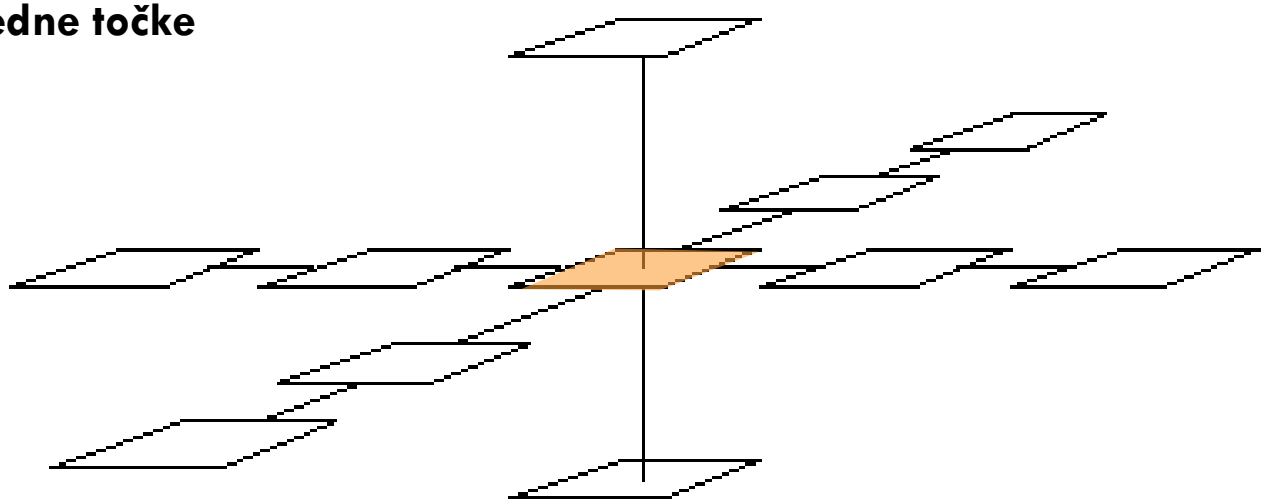
1. FAZA: PODJELA

- Poželjna svojstva
  - sitnozrnata podjela: br_zadataka >> br_procesora
  - bez uvišestručavanja podataka ili računanja
  - zadaci podjednake veličine (~broj instrukcija)
  - podjela skalabilna s veličinom problema
- Razmotriti koji od slijednih algoritama (koji rješavaju isti problem) se najefikasnije može paralelizirati
  - najbolji slijedni algoritam ne mora nužno biti i najpogodniji za paralelizaciju

PRIMJERI

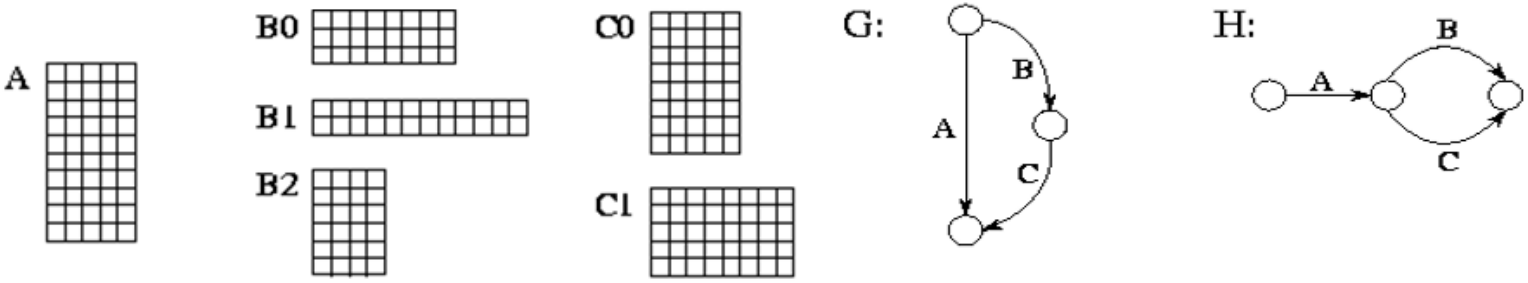
- **Atmosferski model**
  - prostor analize: kvadar podijeljen na diskretne dijelove
  - **podatkovna** podjela: broj zadataka $N_X * N_Y * N_Z$

MASKA za računanje jedne točke

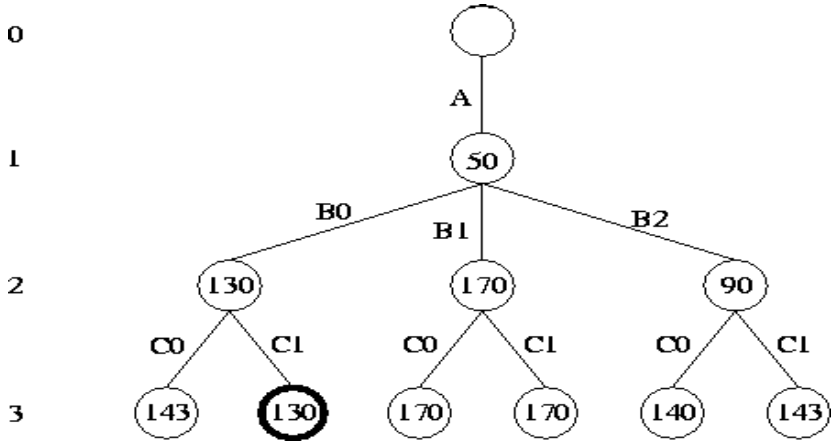


PRIMJERI (2)

- Zauzeće površine



  - **funkcionalna** podjela – čvor u stablu kombinacija → zadatak

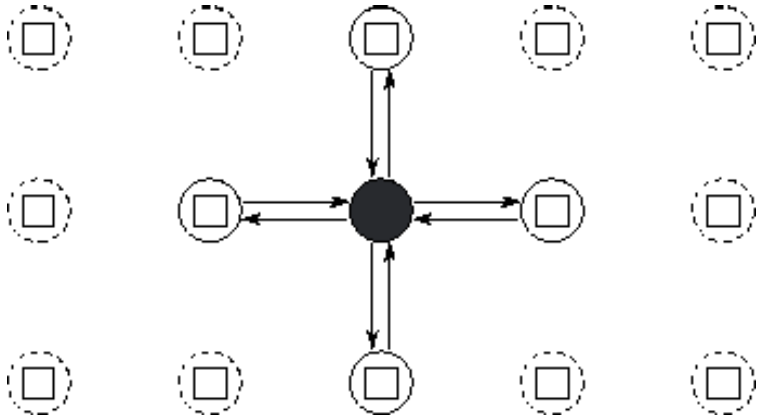


2. FAZA: KOMUNIKACIJA

- Cilj: **raspodjela i smanjenje ukupne količine komunikacije**
  - izbjeći situaciju da samo 1 zadatak šalje i prima
- Vrste komunikacije:
  - **lokalna/globalna**
    - komunicira li zadatak samo sa susjedstvom ili sa ~svima
  - **strukturirana/nestrukturirana**
    - postoji li uzorak (prsten, stablo, hiperkocka…)
  - **statička/dinamička**
    - da li uvijek isti zadaci međusobno komuniciraju ili se određuje tijekom izvođenja (dinamički)
  - **sinkrona/asinkrona**
    - je li unaprijed poznato koji zadaci će komunicirati s kojima

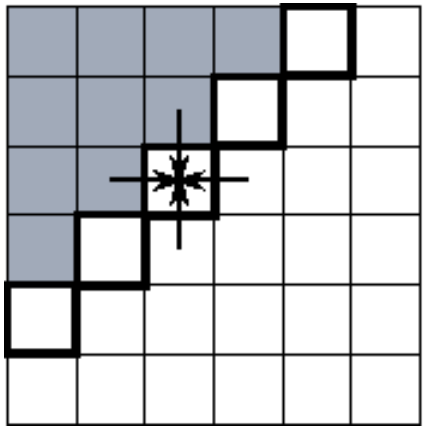
PRIMJER: LOKALNA KOMUNIKACIJA

- Maska (engl. *stencil*) definira **susjedstvo**
- Isplati se optimirati jer najčešće unosi najviše overheada
- **Jacobijeva metoda**
  - nova vrijednost računa se uz pomoć susjednih vrijednosti iz **prethodne** iteracije – svi el. se mogu izračunati u 1 iteraciji
- **Gauss- Seidel metoda**
  - vrijednosti koje kod nekih susjeda odgovaraju ***trenutnoj***, a kod nekih ***prethodnoj*** iteraciji

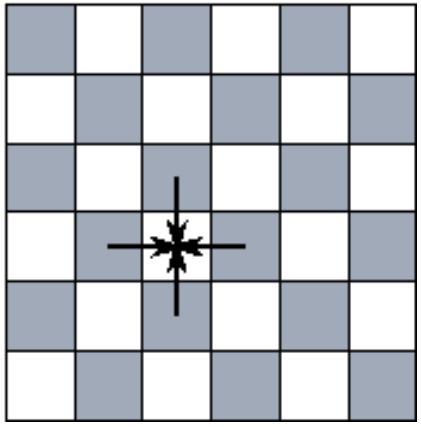


PRIMJER: GAUSS-SEIDEL

- **Varijanta A:** elementi lijevo i gore su iz trenutne, a desno i dolje iz prethodne iteracije



- **Varijanta B:** svi susjedni elementi moraju biti iz trenutne iteracije



https://www.fer.unizg.hr/_download/repository/Simulating%20ocean%20currents.pdf

PRIMJER: GLOBALNA KOMUNIKACIJA

- Prepoznati koji **komunikacijski obrazac** treba upotrijebiti
- Npr. **operacija reduciranja**
  - **Binarno (b-arno) stablo**

```
za(i = 0; (i < log n) && (proces_ID % 2^i == 0); i++)
{ dest_ID = proces_ID XOR 2^i;// bitwise XOR
  ako(proces_ID % 2^(i+1) == 0)
  {  recv(drugi_podatak, dest_ID);
     operacija();           // ovisno o problemu
  }
  inače
     send(moj_podatak, dest_ID);
}
```

KOMUNIKACIJA – POŽELJNA SVOJSTVA

- **Podjednaka** količina komunikacije po zadacima
- Manja količina **lokalne** komunikacije
  - ako puno zadataka mora međusobno razmijeniti informacije, bolje je razmisliti o globalnom obrascu komunikacije
- Mogućnost **istodobne komunikacije**

➔ *Konkretna rješenja za komunikaciju dajemo u idućoj fazi (aglomeracija)*

3. FAZA: AGLOMERACIJA

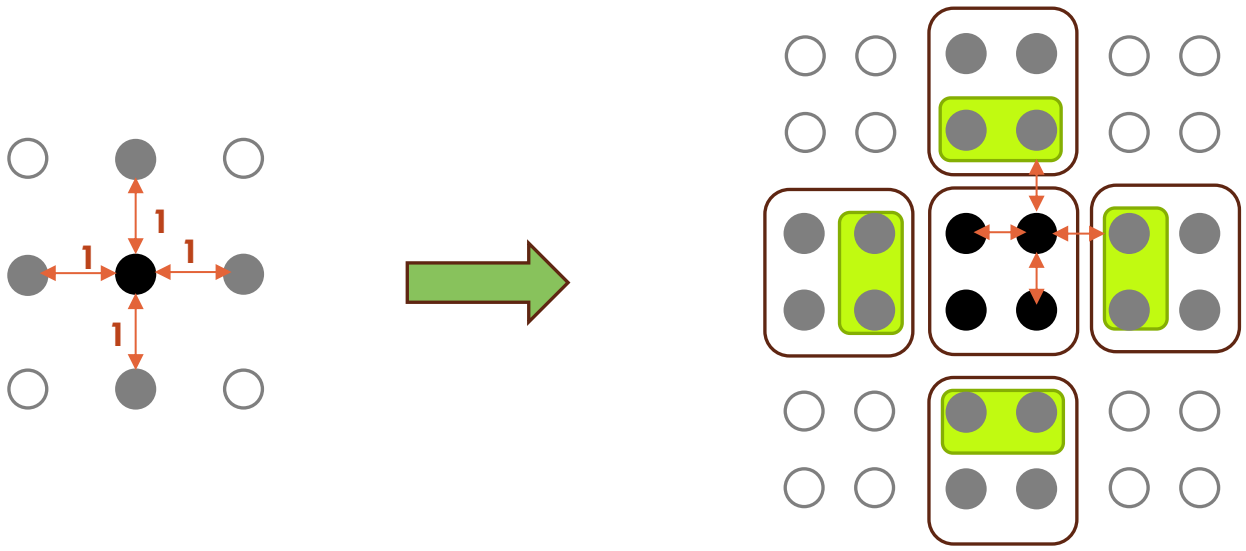
- Ciljevi:
  1. **balansirati** troškove komunikacije i/ili računanja **povećavanjem zrnatosti**
  2. održati **prilagodljivost** programa s obzirom na **veličinu problema**
  3. smanjenje **troškova implementacije** s obzirom na iskoristivost postojećih algoritama
- Ovi ciljevi mogu biti međusobno proturječni

POVEĆANJE ZRNATOSTI

- CILJ: uskladiti **omjer komunikacije i računanja** po zadatku
  - smanjenje komunikacijskih troškova: manji broj poruka i/ili manja količina podataka
- To možemo postići
  1. **Povećavanjem zadataka**
     - grupiranje zadataka (najčešće po svim dimenzijama)
  2. **Uvišestručavanjem računanja**
     - više od jednog zadatka računa isti međurezultat (*pa onda taj rezultat ne moramo komunicirati*)

PRIMJER: POVEĆANJE ZADATAKA

- Elementu *i* su za izračun potrebni podaci od njegova četiri susjeda

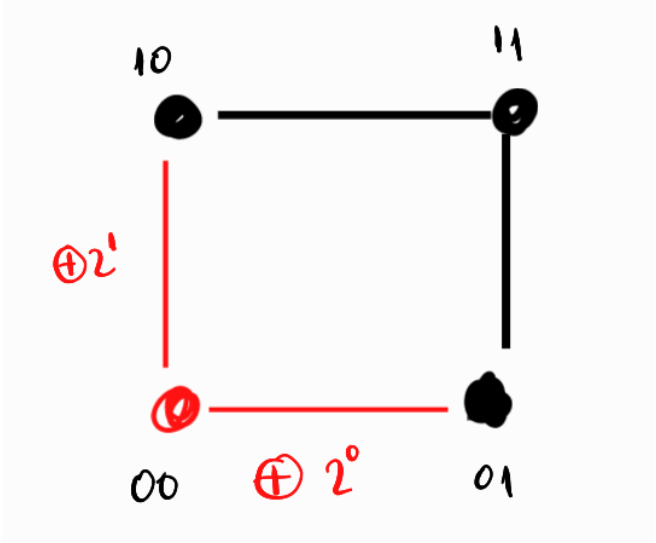


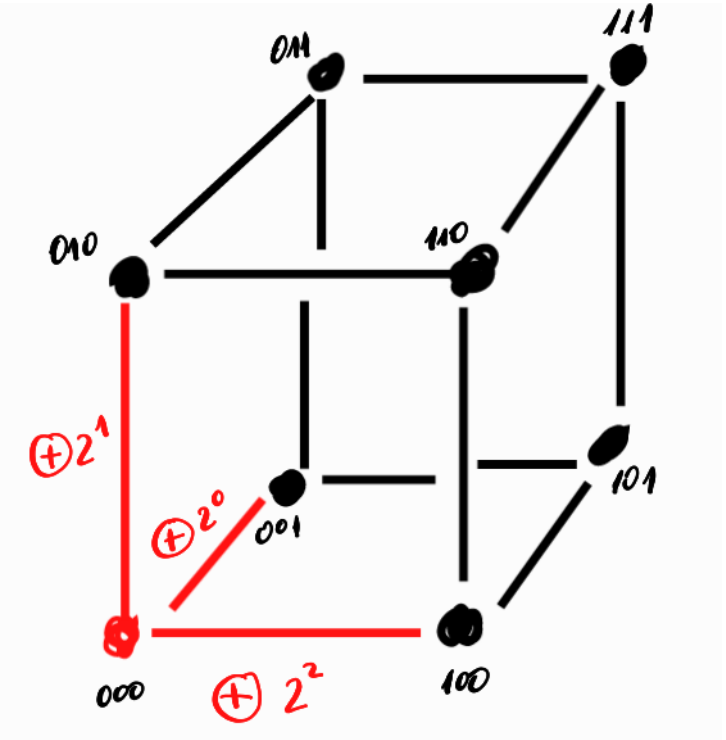
4x manje poruka, u svakoj poruci 2 podatka

PRIMJER: UVIŠESTRUČAVANJE RAČUNANJA

- ALL_REDUCE (svi elementi sadrže rezultat operacije)
  - inicijalno rješenje: 2 prolaza po binarnom stablu → 2logN koraka
  - možemo i efikasnije:

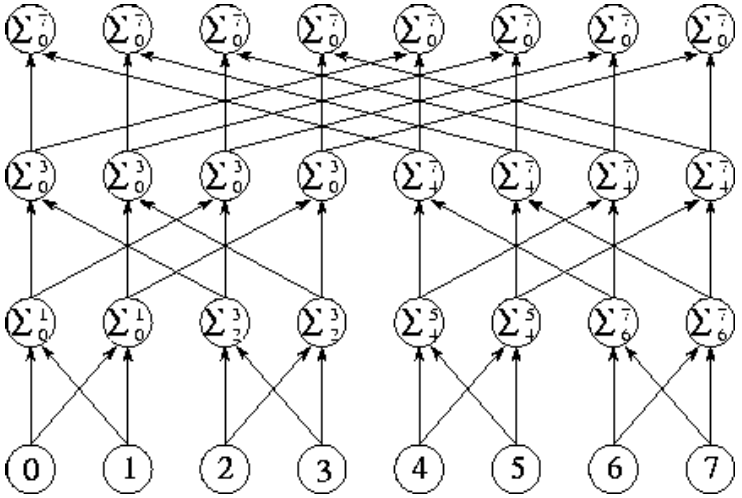
→ HIPERKOCKA





HIPERKOCKA

```
za(i = 0; i < log n; i++)
{ dest_ID = proces_ID XOR 2^i;          // bitwise XOR
  send(moj_podatak, dest_ID);
  recv(drugi_podatak, dest_ID);
  operacija();                          // ovisno o problemu
}
```



logN koraka

PRIMJER: MPI HIPERKOCKA

- Paziti na blokiranje Send i Recv

```
FOR (i = 0; i < log n; i++)
{ dest_ID = proces_ID XOR 2^i;
 IF(ID<dest_ID)
    send(moj_podatak, dest_ID);
    recv(drugi_podatak, dest_ID);
 ELSE
    recv(drugi_podatak, dest_ID);
    send(moj_podatak, dest_ID);
  operacija();
}
```

- U tu svrhu postoji i operacija

```
MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype
sendtype, int dest, int sendtag, void *recvbuf, int recvcount,
MPI_Datatype recvtype, int source, int recvtag, MPI_Comm comm,
MPI_Status * status);
```

OČUVANJE PRILAGODLJIVOSTI

- Ukupan broj zadataka ne smije biti fiksan
- Broj zadataka:
  - ovisan o veličini problema
  - tipično veći od broja procesora

SMANJENJE TROŠKOVA IMPLEMENTACIJE

- U obzir treba uzeti i trošak razvoja paralelnog algoritma (radni sati...), postojeće podatkovne strukture u skladištima podataka i efikasnost pristupa...

AGLOMERACIJA – POŽELJNA SVOJSTVA

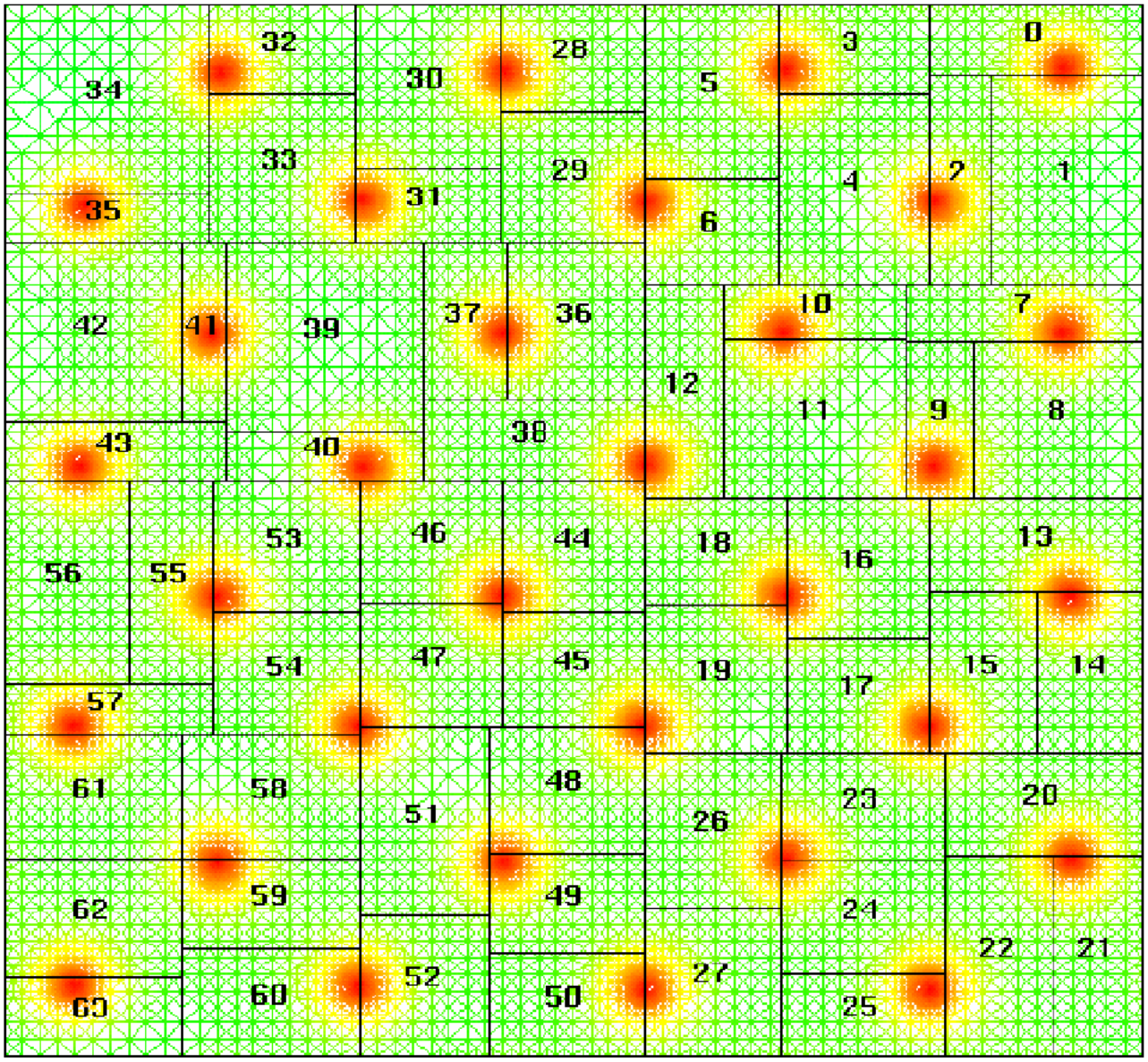
- Podjednaka količina računanja i komunikacije u zadacima
- Prilagodljivost broja zadataka veličini problema
- Što manji broj zadataka (a veći od broja procesora)
- Manji trošak pretvorbe u paralelni oblik

4. PRIDRUŽIVANJE

- Koji procesor će izvoditi koje zadatke?
  - procesor – proces – zadatak
- Najjednostavnije: **jednolika raspodjela** strukture podataka
  - ne mora biti optimalno
  - ovisi o tome koliko je koji zadatak računski zahtjevan
  - tko s kime komunicira
- Pristupi:
  1. **Ujednačavanje opterećenja**
  2. **Raspoređivanje zadataka**

UJEDNAČAVANJE OPTEREĆENJA

- Pretpostavka: **stalan broj zadataka**, svaki zadatak dodijeljen nekom procesoru
- **Statičko**: dodjela samo na početku
- **Dinamičko**: izmjena tijekom rada programa; lokalno i globalno
- Postoji mnogo različitih algoritama
  - A. **Rekurzivna bisekcija**: podjela strukture zadataka:
    - po duljoj dimenziji
    - na temelju računalnih troškova
  - B. **Lokalni algoritmi**: povremena usporedba opterećenja susjednih procesora
    - mali trošak, spora prilagodba
  - C. **Vjerojatnosne metode**: slučajna podjela zadataka po procesorima
    - mali troškovi, radi uvijek; moguća velika komunikacija
  - D. **Cikličko pridruživanje**: P procesora - svakom procesoru svaki P-ti zadatak



http://www.mcs.anl.gov/dbpp/text/pictures/part.gif

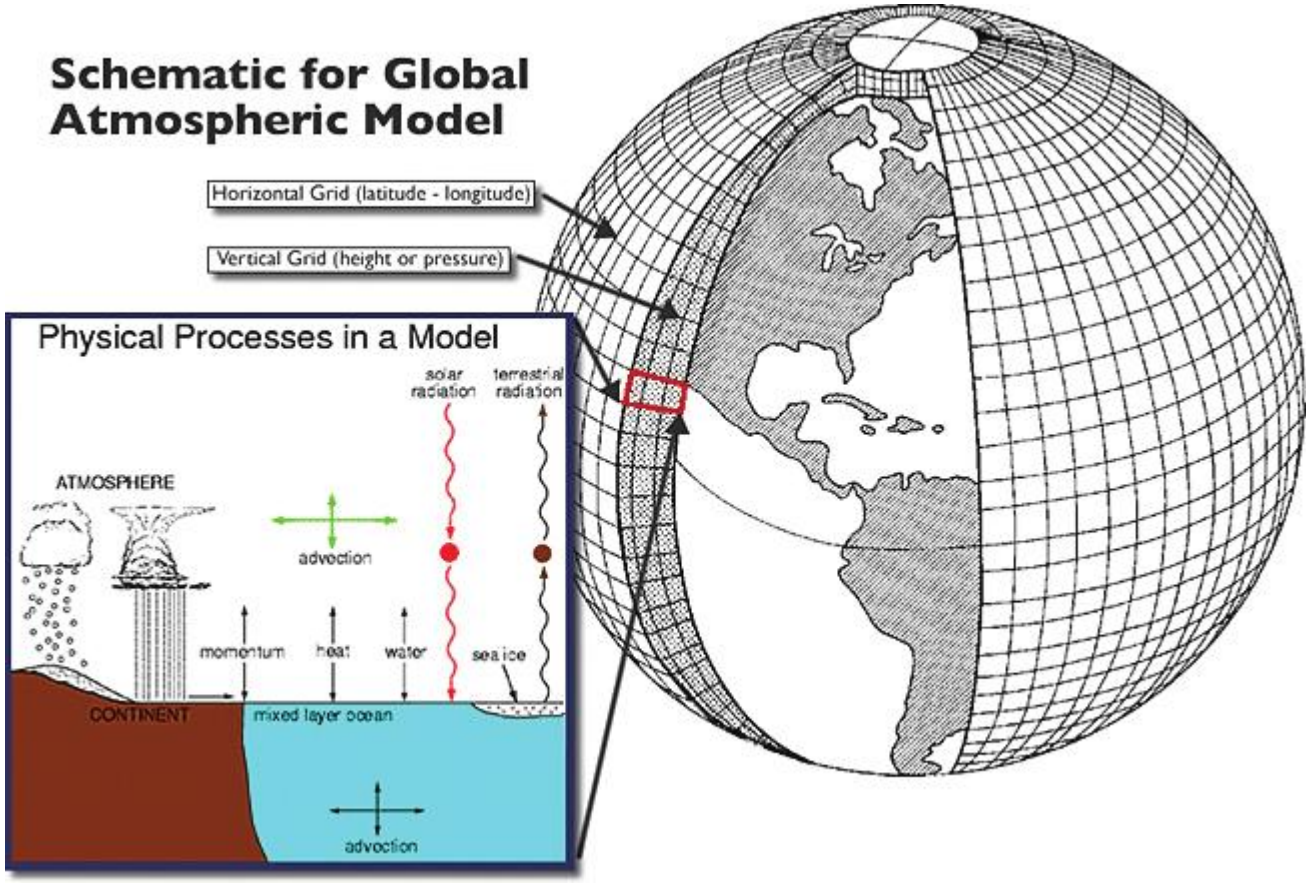
RASPOREĐIVANJE ZADATAKA

- Pretpostavka: više zadataka **kraćeg životnog vijeka** (algoritam je gotov kad su obrađeni svi zadaci)
- Pristupi:
  - **Voditelj/radnik** model (*manager/worker*)
    - proces voditelj dodjeljuje zadatke radnicima
    - potencijalno zagušenje voditelja
  - **Hijerarhijski voditelj/radnik**
    - podgrupe s predradnicima
  - **Decentralizirana metoda**
    - ne postoji zajednički skup zadataka (zadaci se dobivaju od susjeda)
- Potrebno je definirati i mehanizam **otkrivanja završetka** (engl. *termination detection*)

PRIDRUŽIVANJE – POŽELJNA SVOJSTVA

- Isplativi trošak tehnike ujednačavanja
- Voditelj nije preopterećen
- Preduvjeti za određeno pridruživanje

PRIMJER: ATMOSFERSKI MODEL



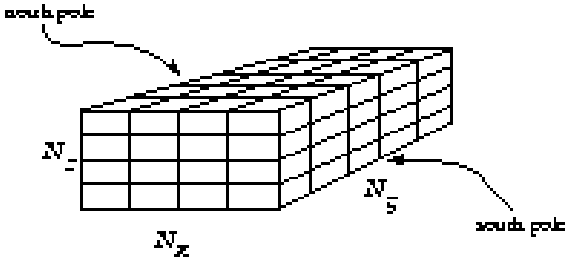
https://www.researchgate.net/figure/Schematic-for-global-atmospheric-model-Source-Center-for-Multiscale-Modeling-of_fig2_323453569

PRIMJER: ATMOSFERSKI MODEL

Model se temelji na 3D mreži: dužina×širina×visina

(vrijednosti tlaka, temp, brzine vjetra, vlažnosti…)

https://www.mcs.anl.gov/~itf/dbpp/text/node20.html#ccm2

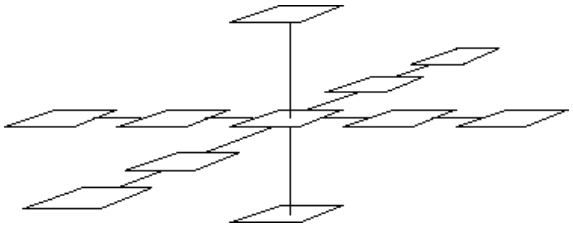


Dvije komponente izračuna:

- **horizontalne interakcije: maska**

  gibanje fluida (npr. advekcija) - parcijalne diferencijalne jednadžbe

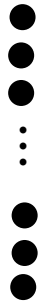
  lokalne ovisnosti



- **vertikalne interakcije: 1D stupac**

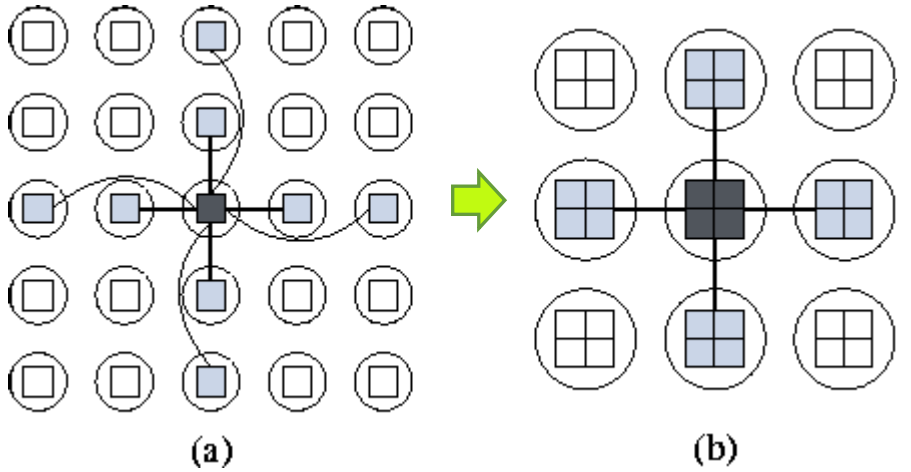
  radijacija, konvekcija - operacije scan, reduce...

  ovisnosti kroz cijeli stupac – za fiksni (x,y) potrebne su informacije iz više (ili svih) razina z



PRIMJER: ATMOSFERSKI MODEL

- Aglomeracija:
  - horizontalno: 2x2



8 poruka s po 1 vrijednosti za izračun 1 elementa

4 poruke s po 4 vrijednosti za izračun 4 elementa

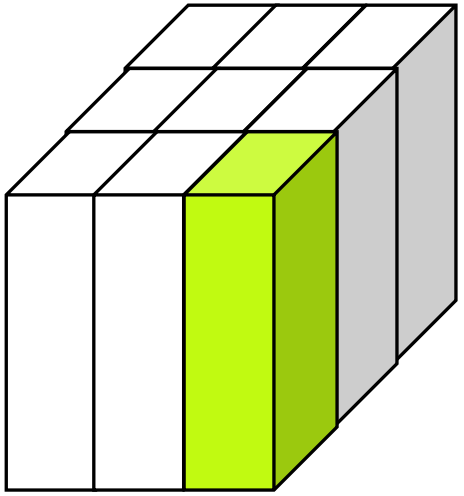
  - vertikalno: grupirati sve zadatke *u jednoj vertikali*

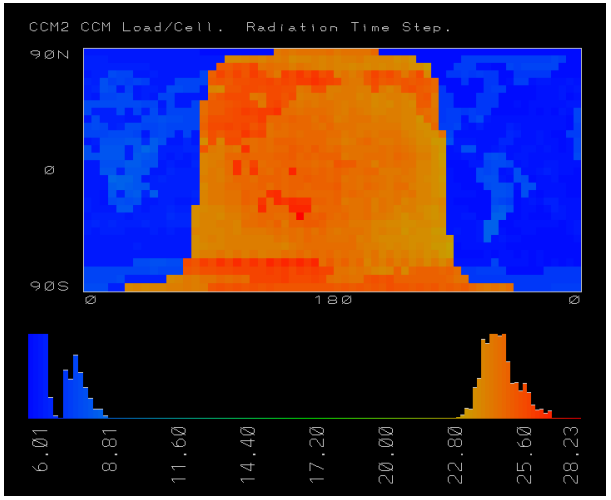
➔ konačno: $(N_X * N_Y)/4$ zadataka

**odabrano 2x2 grupiranje je ilustrativni primjer, optimalno grupiranje ovisi o konkretnim parametrima i nužno se provesti kvantitativnu analizu (6. poglavlje...)*

PRIMJER: ATMOSFERSKI MODEL

- Pridruživanje:
  a) jednolika raspodjela po procesorima
     - prihvatljivo ako vrijedi da je količina računanja podjednaka za sve zadatke
     - međutim, ne računaju se uvijek svi izlazi za sve točke (npr. radijacija se računa samo po danu)





može se donekle poboljšati cikličkim ili nasumičnim pridruživanjem uz minimalni dodatni overhead